

LAPORAN PROJECT
SMART TRAFFIC SIMULATION & SIGNAL OPTIMALIZATION

Disusun untuk memenuhi tugas kelompok mata kuliah Algoritma dan Struktur Data

Dosen pengampu : Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Disusun Oleh:

Kelompok 8

Alwan Fadhilah Rafiyanza	: 25051030077
Faqih Rizky Saputra	: 25051030048
Arwan Adrean M.	: 25051030065
Amanda Lulu Prasetia	: 25051030079

PRODI TEKNIK ELEKTRO
DEPARTMENT PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA

2026

DAFTAR ISI

DAFTAR ISI	2
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
BAB I PENDAHULUAN	6
1.1 Latar Belakang Masalah	6
1.2 Tujuan System.....	6
1.3 Batasan Implementasi.....	7
1.4 Diagram Arsitektur Sistem	7
BAB II PERANCANGAN SISTEM.....	9
2.1 Pemilihan Struktur Data dan Justifikasi	9
2.2 Diagram Interaksi antar Modul	11
2.3 Pseudocode Algoritma Utama	11
2.3.1 Menambahkan Kendaraan ke Antrian Prioritas (Priority Queue)	11
2.3.2 Menentukan Rute Terpendek Menggunakan Dijkstra.....	11
2.3.3 Mengeluarkan Kendaraan dari Antrian	12
2.3.4 Pembuatan Laporan Kemacetan	12
BAB III IMPLEMENTASI	14
3.1 Penjelasan Kode per Modul.....	14
3.1.1 Modul Graph (modul_1_graph.py)	14
3.1.2 Modul Priority Queue (modul_2_priority_queue.py)	14
3.1.3 Modul Dijkstra (modul_3_dijkstra.py).....	14
3.1.4 Modul BST Index (modul_4_bst_index.py).....	15
3.1.5 Modul Sorting (modul_5_sorting.py).....	15
3.1.6 Modul Command Line Interface (CLI) (modul_6_cli.py)	16
3.2 Screenshot CLI Berjalan.....	17
3.3 Contoh Input dan Output Nyata	18
BAB IV ANALISIS BIG-O	30
4.1 Analisis Kompleksitas Waktu dan Ruang Teoritis.....	30
4.2 Tabel Ringkasan	31
4.3 Perbandingan Teoritis vs Eksperimen	32
BAB V HASIL EKSPERIMEN	34

5.1 Pencatatan Waktu Eksekusi (Runtime)	34
5.2 Grafik Tren	34
5.3 Analisis dan Diskusi Hasil Eksperimen	35
BAB VI JAWABAN PERTANYAAN ANALISIS	36
6.1 Analisis Priority Queue Kendaraan	36
6.2 Modifikasi Dijkstra untuk Bobot Dinamis	36
6.3 Perbandingan Merge Sort dan Insertion Sort	37
6.4 Analisis DFS dan Union-Find	37
6.5 Analisis BST Persimpangan	38
BAB VII KESIMPULAN.....	39
7.1 Ringkasan Pencapaian	39
7.2 Refleksi Trade-Off Desain	39
7.3 Saran Pengembangan.....	39
LAMPIRAN	41

DAFTAR GAMBAR

Gambar 1.1 Diagram Arsitektur Sistem	7
Gambar 2.1 Diagram Interaksi antar Modul.....	11
Gambar 3.1 Screenshot CLI	17
Gambar 3.2 Screenshot CLI	17
Gambar 3.3 Screenshot CLI	18
Gambar 5.1 Grafik Tren	34

DAFTAR TABEL

Tabel 4.1 Kompleksitas Waktu dan Ruang Teoritis.....	30
Tabel 4.2 Ringkasan	31
Tabel 4.3 Perbandingan Teoritis vs Eksperimen	32
Tabel 5.1 Pencatatan Waktu Eksekusi.....	34

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Kemacetan lalu lintas merupakan salah satu permasalahan utama pada wilayah perkotaan. Tingginya jumlah kendaraan yang melewati persimpangan jalan dapat menyebabkan keterlambatan perjalanan, pemborosan bahan bakar, serta menurunkan efisiensi transportasi. Untuk mengatasi permasalahan tersebut diperlukan suatu sistem manajemen lalu lintas yang mampu mengelola antrian kendaraan, menentukan rute tercepat, serta melakukan pemantauan kondisi jaringan jalan secara efisien.

Pada proyek ini dikembangkan sebuah aplikasi simulasi lalu lintas berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python. Sistem memanfaatkan beberapa struktur data yaitu Graph untuk merepresentasikan jaringan jalan, Priority Queue untuk mengatur prioritas kendaraan, Binary Search Tree (BST) untuk pencarian data persimpangan, serta Stack untuk menyimpan riwayat perubahan lampu lalu lintas. Dengan mengintegrasikan berbagai struktur data tersebut, sistem dapat melakukan simulasi pengelolaan lalu lintas secara lebih realistis dan efisien.

1.2 Tujuan System

Tujuan dari pengembangan sistem ini adalah:

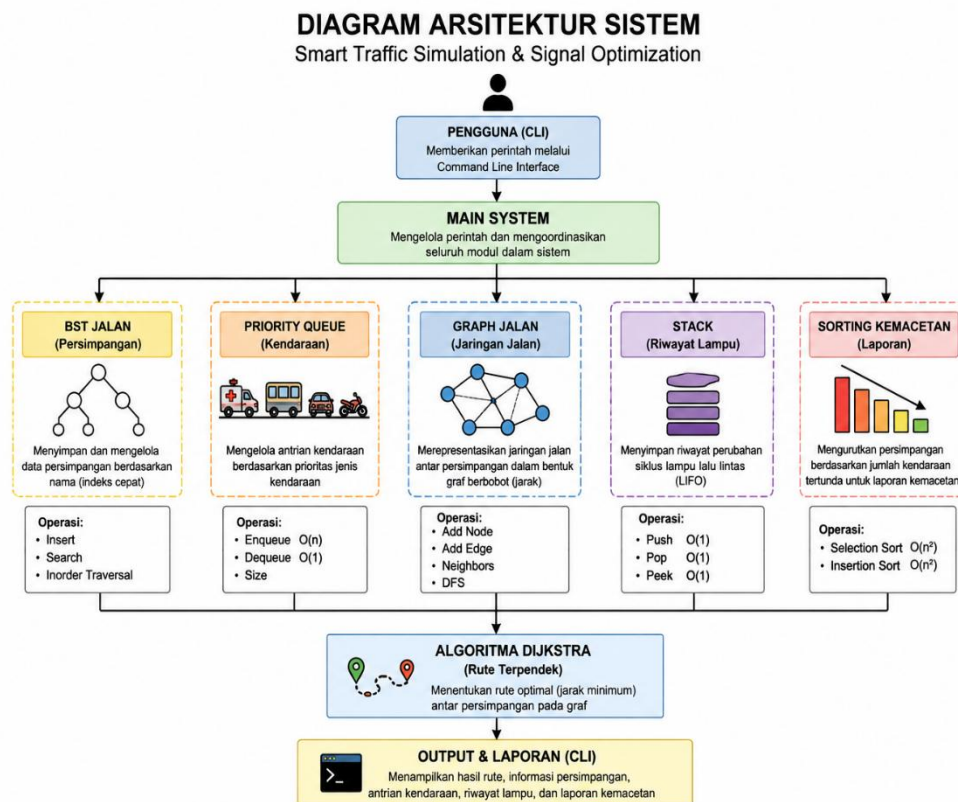
1. Membangun simulasi jaringan lalu lintas menggunakan Graph berbobot.
2. Mengelola antrian kendaraan berdasarkan tingkat prioritas menggunakan Priority Queue.
3. Menentukan rute terpendek antar persimpangan menggunakan algoritma Dijkstra.
4. Menyimpan dan mengelola data persimpangan menggunakan BST.
5. Menyimpan riwayat siklus lampu lalu lintas menggunakan Stack.
6. Melakukan analisis kemacetan melalui proses pengurutan data menggunakan algoritma sorting.

1.3 Batasan Implementasi

Batasan sistem yang diterapkan adalah:

1. Sistem berjalan pada Command Line Interface (CLI).
2. Jumlah persimpangan yang disimulasikan sebanyak 100 node.
3. Jenis kendaraan terdiri dari Ambulans, Bus, Mobil, dan Motor.
4. Prioritas kendaraan ditentukan berdasarkan jenis kendaraan.
5. Algoritma pencarian rute menggunakan Dijkstra tanpa heap.
6. Struktur data diimplementasikan secara manual tanpa library struktur data bawaan Python.

1.4 Diagram Arsitektur Sistem



Gambar 1.1 Diagram Arsitektur Sistem

Berdasarkan diagram arsitektur sistem, proses kerja aplikasi **Smart Traffic Simulation & Signal Optimization** dapat dijelaskan sebagai berikut:

1. Pengguna (CLI)

Pengguna memberikan perintah melalui Command Line Interface (CLI), seperti

menambahkan kendaraan, mencari rute terpendek, melihat antrian kendaraan, atau menampilkan laporan kemacetan.

2. Main System

Perintah yang diberikan pengguna diterima oleh Main System sebagai pusat pengendali aplikasi. Modul ini bertugas mengolah input dan mengarahkan proses ke struktur data atau algoritma yang sesuai.

3. BST Jalan (Persimpangan)

Apabila sistem membutuhkan informasi mengenai suatu persimpangan, pencarian dilakukan menggunakan Binary Search Tree (BST). Struktur data ini digunakan untuk menyimpan dan mengelola data persimpangan sehingga proses pencarian dapat dilakukan lebih cepat.

4. Priority Queue Kendaraan

Ketika kendaraan masuk ke suatu persimpangan, data kendaraan disimpan ke dalam Priority Queue. Antrian ini mengutamakan kendaraan dengan prioritas lebih tinggi, seperti ambulans, sebelum kendaraan lainnya.

5. Graph Jalan (Jaringan Jalan)

Seluruh jaringan jalan direpresentasikan menggunakan Graph berbobot. Setiap persimpangan menjadi node dan setiap jalan menjadi edge yang memiliki bobot berupa jarak antar persimpangan.

6. Stack Riwayat Lampu

Perubahan kondisi lampu lalu lintas disimpan menggunakan Stack. Struktur data ini memungkinkan sistem menyimpan riwayat perubahan lampu berdasarkan prinsip Last In First Out (LIFO).

7. Sorting Kemacetan

Data jumlah kendaraan yang menunggu pada setiap persimpangan diurutkan menggunakan algoritma Selection Sort dan Insertion Sort. Hasil pengurutan digunakan untuk menyusun laporan tingkat kemacetan.

8. Algoritma Dijkstra

Data dari Graph Jalan diproses menggunakan algoritma Dijkstra untuk menentukan rute dengan jarak terpendek dari titik asal ke tujuan yang dipilih pengguna.

9. Output dan Laporan

Setelah seluruh proses selesai, sistem menampilkan hasil kepada pengguna berupa informasi rute terpendek, data persimpangan, jumlah antrian kendaraan, riwayat lampu lalu lintas, dan laporan kemacetan melalui CLI.

BAB II

PERANCANGAN SISTEM

2.1 Pemilihan Struktur Data dan Justifikasi

Pada sistem Smart Traffic Simulation & Signal Optimization, digunakan beberapa struktur data dan algoritma yang dipilih berdasarkan kebutuhan fungsional sistem. Pemilihan setiap struktur data bertujuan untuk meningkatkan efisiensi pengolahan data serta mempermudah implementasi fitur simulasi lalu lintas.

1. Binary Search Tree (BST)

BST digunakan untuk menyimpan dan mengelola data persimpangan berdasarkan nama persimpangan. Struktur data ini dipilih karena mampu melakukan operasi pencarian dan penyisipan data dengan kompleksitas rata-rata $O(\log n)$. Dengan BST, informasi persimpangan dapat ditemukan lebih cepat dibandingkan pencarian secara linear, terutama ketika jumlah persimpangan semakin banyak.

2. Priority Queue

Priority Queue digunakan untuk mengatur antrian kendaraan pada setiap persimpangan. Struktur data ini dipilih karena sistem membutuhkan mekanisme pelayanan berdasarkan tingkat prioritas kendaraan. Kendaraan darurat seperti ambulans harus mendapatkan prioritas lebih tinggi dibandingkan bus, mobil, atau motor. Priority Queue memungkinkan kendaraan dengan prioritas tertinggi diproses terlebih dahulu sehingga sesuai dengan kebutuhan simulasi lalu lintas.

3. Stack

Stack digunakan untuk menyimpan riwayat perubahan kondisi lampulalu lintas. Struktur data ini dipilih karena menerapkan prinsip Last In First Out (LIFO), sehingga perubahan kondisi lampu yang paling baru dapat diakses dengan cepat melalui operasi peek atau pop. Implementasi Stack mempermudah pencatatan dan penelusuran riwayat siklus lampu lalu lintas.

4. Graph

Graph digunakan untuk merepresentasikan jaringan jalan antar persimpangan. Setiap persimpangan direpresentasikan sebagai vertex (node), sedangkan jalan penghubung direpresentasikan sebagai edge yang memiliki bobot berupa jarak. Struktur data Graph dipilih karena mampu menggambarkan hubungan antar lokasi secara alami serta mendukung implementasi algoritma pencarian rute terpendek.

5. Linked List

Linked List digunakan sebagai dasar implementasi beberapa struktur data dalam sistem, seperti Priority Queue, Stack, dan adjacency list pada Graph. Struktur data ini dipilih karena memiliki ukuran yang dinamis dan memudahkan proses penyisipan maupun penghapusan data tanpa perlu menggeser elemen lain seperti pada array.

6. Algoritma Dijkstra

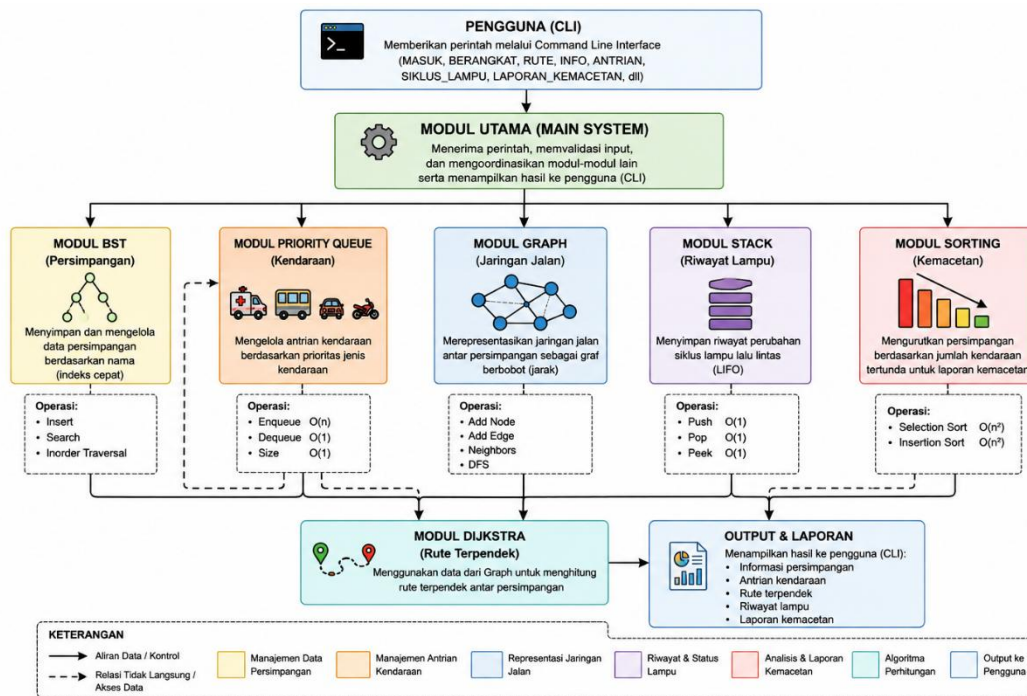
Algoritma Dijkstra dipilih untuk menentukan rute terpendek antara dua persimpangan pada jaringan jalan. Algoritma ini mampu menghitung jarak minimum dari titik asal ke titik tujuan berdasarkan bobot jarak pada Graph sehingga cocok digunakan dalam simulasi navigasi kendaraan.

7. Algoritma Sorting (Selection Sort dan Insertion Sort)

Selection Sort dan Insertion Sort digunakan untuk mengurutkan data jumlah kendaraan pada setiap persimpangan dalam laporan kemacetan. Kedua algoritma tersebut dipilih karena mudah diimplementasikan dan sesuai untuk kebutuhan analisis serta perbandingan performa algoritma pengurutan pada skala data yang tidak terlalu besar.

Berdasarkan pemilihan struktur data dan algoritma tersebut, sistem mampu mengelola antrian kendaraan, menyimpan data persimpangan, menentukan rute optimal, mencatat riwayat lampu lalu lintas, serta menghasilkan laporan kemacetan secara efektif sesuai kebutuhan simulasi lalu lintas yang dirancang.

2.2 Diagram Interaksi antar Modul



Gambar 2.1 Diagram Interaksi antar Modul

2.3 Pseudocode Algoritma Utama

2.3.1 Menambahkan Kendaraan ke Antrian Prioritas (Priority Queue)

Algoritma ini digunakan ketika kendaraan memasuki persimpangan. Kendaraan akan ditempatkan pada posisi yang sesuai berdasarkan tingkat prioritasnya.
Pseudocode:

1. ALGORITMA MASUK_KENDARAAN

Input : data kendaraan

Output : kendaraan masuk ke antrian prioritas

Buat node kendaraan baru

IF antrian kosong THEN

 jadikan node sebagai head

ELSE IF prioritas kendaraan lebih tinggi dari head THEN

 sisipkan node di depan

ELSE telusuri antrian hingga posisi prioritas sesuai

 sisipkan node pada posisi tersebut

END IF

Tambah jumlah kendaraan dalam antrian

2.3.2 Menentukan Rute Terpendek Menggunakan Dijkstra

Algoritma Dijkstra digunakan untuk menentukan jalur dengan jarak minimum antara persimpangan asal dan tujuan.

Pseudocode:

1. ALGORITMA DIJKSTRA

Input : graph, node asal

Output : jarak minimum dan jalur terpendek

Untuk setiap node

 jarak[node] ← tak hingga

 parent[node] ← kosong

jarak[asal] ← 0

Selama masih ada node yang belum dikunjungi

 pilih node dengan jarak terkecil

 tandai sebagai dikunjungi

 untuk setiap tetangga node

 jika jarak baru lebih kecil

 perbarui jarak

 simpan parent

 endif

 endfor

endwhile

kembalikan jarak dan parent

2.3.3 Mengeluarkan Kendaraan dari Antrian

Algoritma ini digunakan ketika kendaraan mendapatkan giliran untuk melintas pada persimpangan.

Pseudocode:

1. ALGORITMA BERANGKAT_KENDARAAN

Input : antrian kendaraan

Output : kendaraan prioritas tertinggi keluar

IF antrian kosong **THEN**

 tampilkan "Antrian kosong"

ELSE

 ambil kendaraan pada head

 pindahkan head ke node berikutnya

 kurangi ukuran antrian

 tampilkan data kendaraan

END IF

2.3.4 Pembuatan Laporan Kemacetan

Algoritma ini digunakan untuk mengurutkan persimpangan berdasarkan jumlah kendaraan yang sedang mengantre.

Pseudocode:

ALGORITMA LAPORAN_KEMACETAN

Input : daftar persimpangan dan jumlah antrian

Output : data terurut menurun

```
FOR i ← 0 sampai n-1
    indeks_maks ← i
    FOR j ← i+1 sampai n
        IF data[j] lebih besar THEN indeks_maks ← j END IF
    ENDFOR
    tukar data[i] dengan data[indeks_maks]
ENDFOR
tampilkan hasil
```

BAB III

IMPLEMENTASI

3.1 Penjelasan Kode per Modul

3.1.1 Modul Graph (modul_1_graph.py)

Modul Graph digunakan untuk melakukan analisis konektivitas jaringan jalan. Fungsi utama pada modul ini adalah `cek_isolasi()`, yang berfungsi mendeteksi persimpangan yang tidak terhubung dengan jaringan jalan utama.

```
def cek_isolasi(graph):  
    start = next(iter(graph.adj))  
    visited = graph.dfs(start)  
  
    return [  
        v  
        for v in graph.adj if v not in visited  
    ]
```

Fungsi ini memanfaatkan algoritma Depth First Search (DFS) yang terdapat pada objek `GraphJalan`. Proses dimulai dari satu simpul awal, kemudian menelusuri seluruh simpul yang dapat dicapai. Setelah proses DFS selesai, simpul yang tidak termasuk dalam himpunan `visited` dianggap sebagai persimpangan terisolasi dan dikembalikan sebagai hasil fungsi. Fungsi utama yaitu mendeteksi persimpangan yang tidak terhubung dengan jaringan jalan.

3.1.2 Modul Priority Queue (modul_2_priority_queue.py)

Modul Priority Queue digunakan untuk mengelola data antrian kendaraan pada setiap persimpangan.

```
def jumlah_antrian(queue):  
    return len(queue)
```

Fungsi `jumlah_antrian()` digunakan untuk memperoleh jumlah kendaraan yang sedang berada dalam antrian. Fungsi ini memanfaatkan method `__len__()` pada struktur data Priority Queue sehingga proses penghitungan jumlah kendaraan dapat dilakukan secara cepat. Fungsi utama yaitu Menampilkan jumlah kendaraan yang sedang mengantre pada suatu persimpangan.

3.1.3 Modul Dijkstra (modul_3_dijkstra.py)

Modul ini digunakan untuk menentukan rute terpendek antar persimpangan menggunakan algoritma Dijkstra.

1. Fungsi utama pertama adalah `dijkstra_rute()`.

```
def dijkstra_rute(graph, asal):
```

Fungsi ini menghitung jarak minimum dari titik asal menuju seluruh simpul dalam graph. Algoritma bekerja dengan memilih simpul yang memiliki jarak terkecil dan memperbarui jarak ke simpul tetangganya hingga seluruh simpul dikunjungi.

Hasil yang diperoleh berupa:

- dist : menyimpan jarak minimum menuju setiap simpul.
- parent : menyimpan simpul sebelumnya untuk membentuk jalur.

2. Fungsi kedua adalah ambil_jalur().

```
def ambil_jalur(parent, tujuan):
```

Fungsi ini digunakan untuk membangun kembali jalur terpendek dari data parent yang dihasilkan algoritma Dijkstra. Jalur dibentuk mulai dari simpul tujuan hingga kembali ke simpul asal, kemudian urutannya dibalik agar sesuai dengan arah perjalanan. Fungsi utama yaitu menentukan rute dan jarak terpendek antar persimpangan.

3.1.4 Modul BST Index (modul_4_bst_index.py)

Modul BST digunakan untuk melakukan pencarian data persimpangan secara efisien menggunakan Binary Search Tree.

```
def cari_persimpangan(bst, nama):  
    return bst.search(nama)
```

Fungsi cari_persimpangan() menerima objek BST dan nama persimpangan sebagai parameter. Proses pencarian dilakukan menggunakan method search() pada BST sehingga data persimpangan dapat ditemukan lebih cepat dibandingkan pencarian secara linear. Fungsi utama yaitu mencari informasi persimpangan berdasarkan nama.

3.1.5 Modul Sorting (modul_5_sorting.py)

Modul Sorting digunakan untuk mengurutkan data kemacetan berdasarkan jumlah kendaraan yang berada pada setiap persimpangan.

Selection Sort

```
def selection_sort(data):
```

Algoritma Selection Sort bekerja dengan mencari elemen terbesar pada bagian data yang belum terurut, kemudian menukarnya dengan posisi yang sesuai. Proses ini dilakukan berulang hingga seluruh data terurut secara menurun.

Insertion Sort

```
def insertion_sort(data):
```

Algoritma Insertion Sort bekerja dengan mengambil satu elemen kemudian menyisipkannya ke posisi yang tepat pada bagian data yang sudah terurut sebelumnya. Proses ini dilakukan hingga seluruh elemen tersusun secara menurun berdasarkan jumlah kendaraan. Fungsi utama mengurutkan tingkat kemacetan pada setiap persimpangan.

3.1.6 Modul Command Line Interface (CLI) (modul_6_cli.py)

Modul CLI berfungsi sebagai antarmuka pengguna berbasis teks yang menyediakan informasi mengenai perintah-perintah yang dapat dijalankan pada sistem.

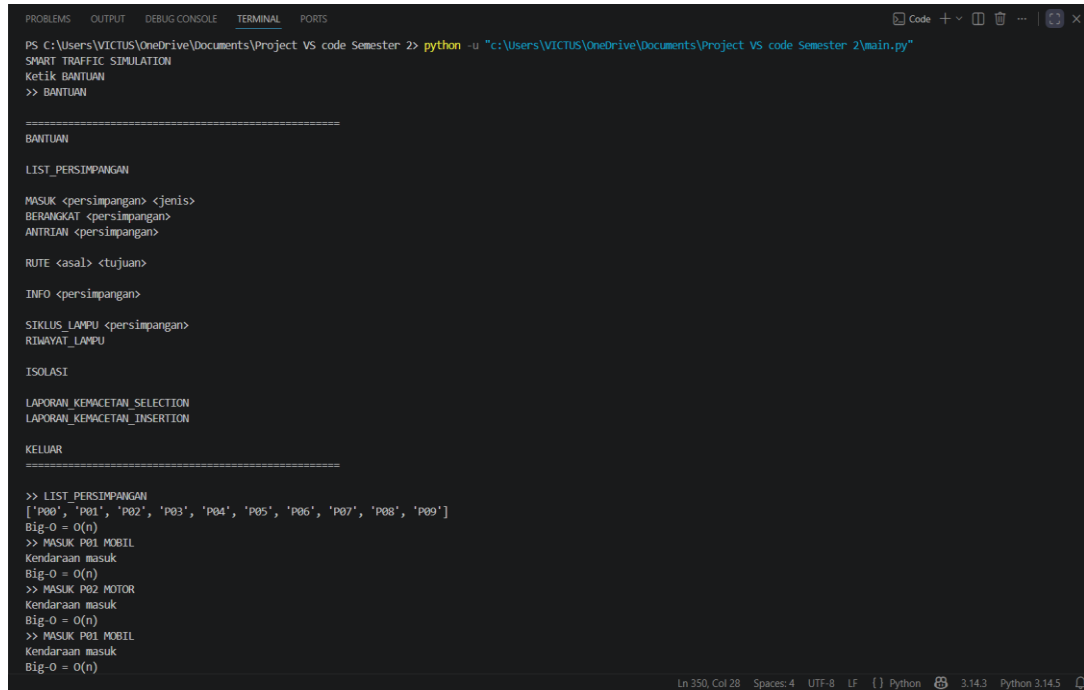
```
def tampil_bantuan():
```

Fungsi ini menampilkan daftar seluruh perintah yang tersedia, seperti:

- LIST_PERSIMPANGAN
- MASUK
- BERANGKAT
- ANTRIAN
- RUTE
- INFO
- SIKLUS_LAMPU
- RIWAYAT_LAMPU
- ISOLASI
- LAPORAN_KEMACETAN_SELECTION
- LAPORAN_KEMACETAN_INSERTION
- KELUAR

Modul ini membantu pengguna memahami cara mengoperasikan sistem tanpa harus melihat kode program secara langsung. Fungsi utama yaitu menampilkan panduan penggunaan sistem dan daftar perintah yang tersedia.

3.2 Screenshot CLI Berjalan



```
PS C:\Users\VICTUS\OneDrive\Documents\Project VS code Semester 2> python -u "c:\Users\VICTUS\OneDrive\Documents\Project VS code Semester 2\main.py"
SMART TRAFFIC SIMULATION
Ketik BANTUAN
>> BANTUAN

=====
BANTUAN

LIST_PERSIMPANGAN

MASUK <persimpangan> <jenis>
BERANGKAT <persimpangan>
ANTRIAN <persimpangan>

RUTE <asal> <tujuan>

INFO <persimpangan>

SIKLUS_LAMPU <persimpangan>
RIWAYAT_LAMPU

ISOLASI

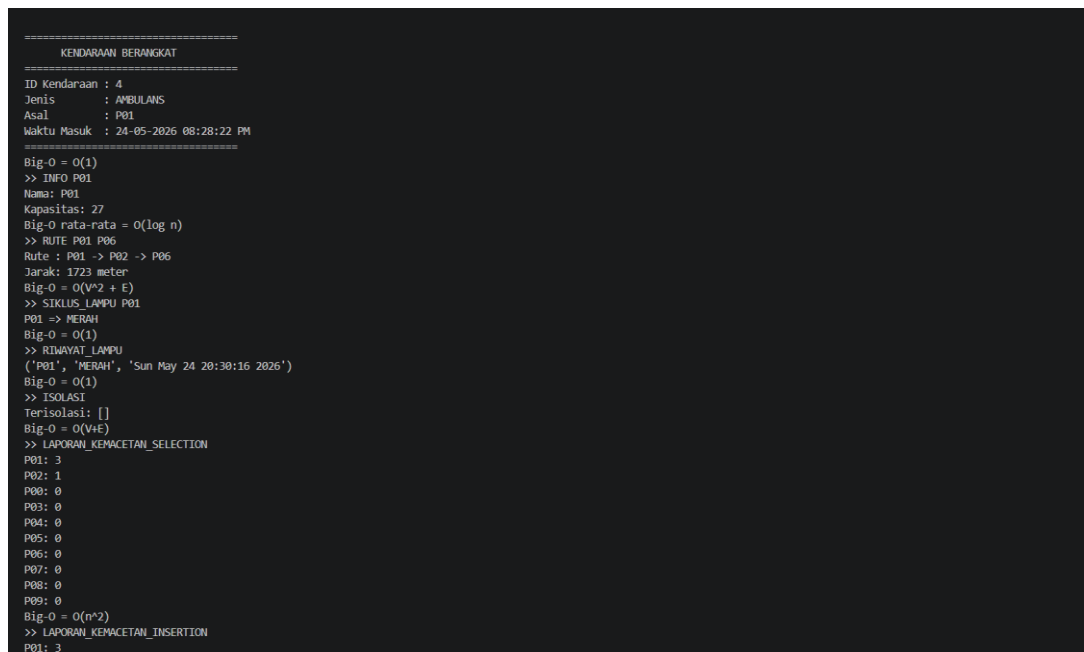
LAPORAN_KEMACETAN_SELECTION
LAPORAN_KEMACETAN_INSERTION

KELUAR

=====

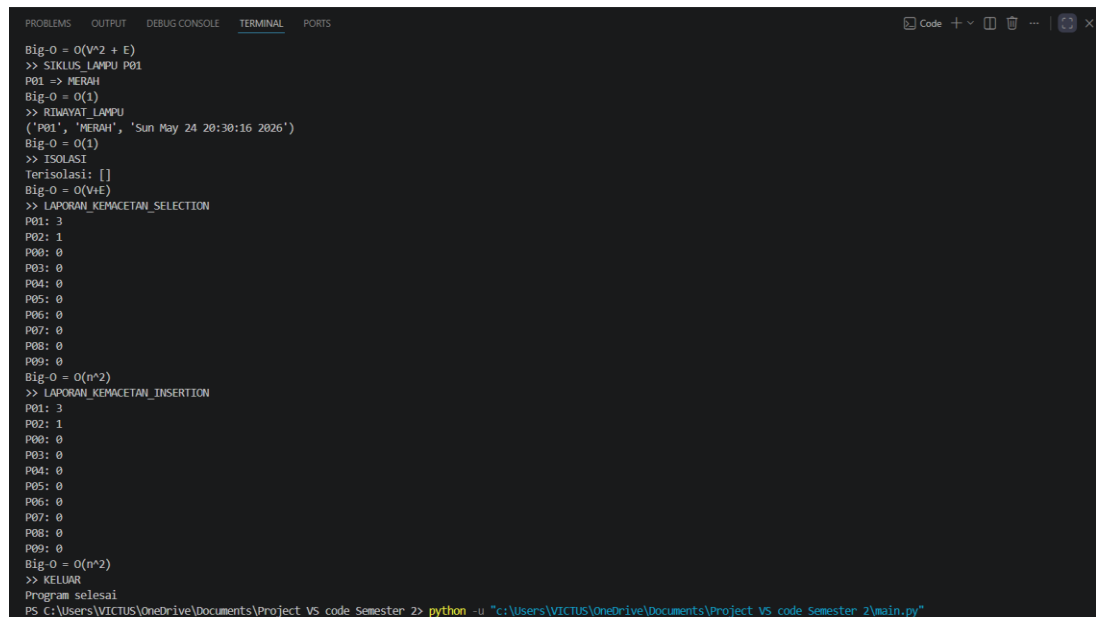
>> LIST_PERSIMPANGAN
['P00', 'P01', 'P02', 'P03', 'P04', 'P05', 'P06', 'P07', 'P08', 'P09']
Big-O = O(n)
>> MASUK P01 MOBIL
Kendaraan masuk
Big-O = O(n)
>> MASUK P02 MOTOR
Kendaraan masuk
Big-O = O(n)
>> MASUK P01 MOBIL
Kendaraan masuk
Big-O = O(n)
```

Gambar 3.1 Screenshot CLI



```
=====
KENDARAAN BERANGKAT
=====
ID Kendaraan : 4
Jenis : AMBULANS
Asal : P01
Waktu Masuk : 24-05-2026 08:28:22 PM
=====
Big-O = O(1)
>> INFO P01
Nama: P01
Kapasitas: 27
Big-O rata-rata = O(log n)
>> RUTE P01 P06
Rute : P01 -> P02 -> P06
Jarak: 1723 meter
Big-O = O(V^2 + E)
>> SIKLUS_LAMPU P01
P01 => MERAH
Big-O = O(1)
>> RIWAYAT_LAMPU
('P01', 'MERAH', 'Sun May 24 20:30:16 2026')
Big-O = O(1)
>> ISOLASI
Terisolasi: []
Big-O = O(V+E)
>> LAPORAN_KEMACETAN_SELECTION
P01: 3
P02: 1
P03: 0
P04: 0
P05: 0
P06: 0
P07: 0
P08: 0
P09: 0
Big-O = O(n^2)
>> LAPORAN_KEMACETAN_INSERTION
P01: 3
```

Gambar 3.2 Screenshot CLI



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Big-O = O(V^2 + E)
>> SIKLUS_LAMPU P01
P01 => MERAH
Big-O = O(1)
>> RIWAYAT_LAMPU
('P01', 'MERAH', 'Sun May 24 20:30:16 2026')
Big-O = O(1)
>> ISOLASI
Terisolasi: []
Big-O = O(V+E)
>> LAPORAN_KEMACETAN_SELECTION
P01: 3
P02: 1
P00: 0
P03: 0
P04: 0
P05: 0
P06: 0
P07: 0
P08: 0
P09: 0
Big-O = O(n^2)
>> LAPORAN_KEMACETAN_INSERTION
P01: 3
P02: 1
P00: 0
P03: 0
P04: 0
P05: 0
P06: 0
P07: 0
P08: 0
P09: 0
Big-O = O(n^2)
>> KELUAR
Program selesai
PS C:\Users\VICTUS\OneDrive\Documents\Project VS code Semester 2> python -u "c:\Users\VICTUS\OneDrive\Documents\Project VS code Semester 2\main.py"
```

Gambar 3.3 Screenshot CLI

3.3 Contoh Input dan Output Nyata

Input:

```
1. import numpy as np
2. import random
3. import time
4. from dataclasses import dataclass
5.
6. np.random.seed(17)
7. random.seed(17)
8.
9. JENIS_KENDARAAN = {
10.     "AMBULANS": 1,
11.     "BUS": 2,
12.     "MOBIL": 3,
13.     "MOTOR": 4
14. }
15.
16. NAMA_JENIS = {
17.     1: "AMBULANS",
18.     2: "BUS",
19.     3: "MOBIL",
20.     4: "MOTOR"
21. }
22.
23. # ===== DATA =====
24.
25. @dataclass
26. class Kendaraan:
27.     id_kendaraan: int
28.     jenis: int
29.     asal: str
30.     tujuan: str
31.     waktu_masuk: str
```

```

32.
33. @dataclass
34. class Persimpangan:
35.     nama: str
36.     kapasitas: int = 20
37.
38. # ===== LINKED LIST =====
39.
40. class LLNode:
41.     def __init__(self, data=None):
42.         self.data = data
43.         self.next = None
44.
45. # ===== PRIORITY QUEUE =====
46.
47. class PriorityQueueKendaraan:
48.
49.     def __init__(self):
50.         self.head = None
51.         self._size = 0
52.
53.     def enqueue(self, kendaraan):
54.         node = LLNode(kendaraan)
55.
56.         if self.head is None:
57.             self.head = node
58.
59.         elif kendaraan.jenis < self.head.data.jenis:
60.             node.next = self.head
61.             self.head = node
62.
63.         else:
64.             curr = self.head
65.
66.             while (curr.next and
67.                   curr.next.data.jenis <= kendaraan.jenis):
68.                 curr = curr.next
69.
70.             node.next = curr.next
71.             curr.next = node
72.
73.             self._size += 1
74.
75.     def dequeue(self):
76.
77.         if self.head is None:
78.             return None
79.
80.         data = self.head.data
81.         self.head = self.head.next
82.         self._size -= 1
83.
84.         return data
85.
86.     def __len__(self):
87.         return self._size
88.
89. # ===== STACK =====
90.
91. class Stack:
92.
93.     def __init__(self):
94.         self.top = None
95.         self._size = 0

```

```

96.
97.     def push(self, data):
98.         node = LLNode(data)
99.         node.next = self.top
100.        self.top = node
101.        self._size += 1
102.
103.    def pop(self):
104.
105.        if self.top is None:
106.            return None
107.
108.        data = self.top.data
109.        self.top = self.top.next
110.        self._size -= 1
111.
112.        return data
113.
114.    def peek(self):
115.        return self.top.data if self.top else None
116.
117.    # ===== GRAPH =====
118.
119.    class EdgeNode:
120.
121.        def __init__(self, dest, jarak, lajur):
122.            self.dest = dest
123.            self.jarak = jarak
124.            self.lajur = lajur
125.            self.next = None
126.
127.    class GraphJalan:
128.
129.        def __init__(self):
130.            self.adj = {}
131.            self.persimpangan = {}
132.
133.        def tambah_persimpangan(self, p):
134.            self.persimpangan[p.nama] = p
135.            self.adj[p.nama] = None
136.
137.        def tambah_jalan(self, asal, tujuan, jarak, lajur, dua_arah=True):
138.
139.            n = EdgeNode(tujuan, jarak, lajur)
140.            n.next = self.adj[asal]
141.            self.adj[asal] = n
142.
143.            if dua_arah:
144.                n2 = EdgeNode(asal, jarak, lajur)
145.                n2.next = self.adj[tujuan]
146.                self.adj[tujuan] = n2
147.
148.        def tetangga(self, nama):
149.
150.            hasil = []
151.            curr = self.adj[nama]
152.
153.            while curr:
154.                hasil.append((curr.dest, curr.jarak))
155.                curr = curr.next
156.
157.            return hasil
158.
159.        def degree(self, nama):

```

```

160.
161.     d = 0
162.     curr = self.adj[nama]
163.
164.     while curr:
165.         d += 1
166.         curr = curr.next
167.
168.     return d
169.
170. def dfs(self, start):
171.
172.     visited = set()
173.
174.     def _dfs(v):
175.
176.         visited.add(v)
177.
178.         curr = self.adj[v]
179.
180.         while curr:
181.             if curr.dest not in visited:
182.                 _dfs(curr.dest)
183.             curr = curr.next
184.
185.     _dfs(start)
186.
187.     return visited
188.
189. # ===== BST =====
190.
191. class BSTNode:
192.
193.     def __init__(self, nama, p):
194.         self.nama = nama
195.         self.p = p
196.         self.left = None
197.         self.right = None
198.
199. class BSTJalan:
200.
201.     def __init__(self):
202.         self.root = None
203.
204.     def insert(self, nama, p):
205.
206.         def _insert(root):
207.
208.             if root is None:
209.                 return BSTNode(nama, p)
210.
211.             if nama < root.nama:
212.                 root.left = _insert(root.left)
213.             else:
214.                 root.right = _insert(root.right)
215.
216.             return root
217.
218.         self.root = _insert(self.root)
219.
220.     def search(self, nama):
221.
222.         curr = self.root
223.

```

```

224.         while curr:
225.
226.             if nama == curr.nama:
227.                 return curr.p
228.
229.             if nama < curr.nama:
230.                 curr = curr.left
231.             else:
232.                 curr = curr.right
233.
234.         return None
235.
236.     def inorder(self):
237.
238.         hasil = []
239.
240.         def _in(node):
241.
242.             if node:
243.                 _in(node.left)
244.                 hasil.append(node.nama)
245.                 _in(node.right)
246.
247.         _in(self.root)
248.
249.         return hasil
250.
251. # ===== DIJKSTRA =====
252.
253. def dijkstra_rute(graph, asal):
254.
255.     INF = float("inf")
256.
257.     dist = {v: INF for v in graph.adj}
258.     parent = {v: None for v in graph.adj}
259.
260.     visited = set()
261.     dist[asal] = 0
262.
263.     while len(visited) < len(graph.adj):
264.
265.         u = min(
266.             (v for v in graph.adj if v not in visited),
267.             key=lambda x: dist[x]
268.         )
269.
270.         visited.add(u)
271.
272.         curr = graph.adj[u]
273.
274.         while curr:
275.
276.             if dist[u] + curr.jarak < dist[curr.dest]:
277.                 dist[curr.dest] = dist[u] + curr.jarak
278.                 parent[curr.dest] = u
279.
280.             curr = curr.next
281.
282.         return dist, parent
283.
284. def ambil_jalur(parent, tujuan):
285.
286.     path = []
287.     curr = tujuan

```

```

288.
289.     while curr:
290.         path.append(curr)
291.         curr = parent[curr]
292.
293.     path.reverse()
294.
295.     return path
296.
297. # ===== SORTING =====
298.
299. def selection_sort(data):
300.
301.     arr = data[:]
302.
303.     n = len(arr)
304.
305.     for i in range(n):
306.
307.         idx = i
308.
309.         for j in range(i + 1, n):
310.
311.             if arr[j][1] > arr[idx][1]:
312.                 idx = j
313.
314.         arr[i], arr[idx] = arr[idx], arr[i]
315.
316.     return arr
317.
318. def insertion_sort(data):
319.
320.     arr = data[:]
321.
322.     for i in range(1, len(arr)):
323.
324.         key = arr[i]
325.         j = i - 1
326.
327.         while j >= 0 and arr[j][1] < key[1]:
328.             arr[j + 1] = arr[j]
329.             j -= 1
330.
331.         arr[j + 1] = key
332.
333.     return arr
334.
335. # ===== UTIL =====
336.
337. def generate_jaringan(n=100, seed=17):
338.
339.     rng = np.random.default_rng(seed)
340.
341.     nama_p = [f"P{i:02d}" for i in range(n)]
342.
343.     persimpangan = [
344.         Persimpangan(nm, int(rng.integers(15, 30)))
345.         for nm in nama_p
346.     ]
347.
348.     perm = rng.permutation(n)
349.
350.     edges = []
351.

```

```

352.     for i in range(1, n):
353.
354.         u = nama_p[perm[i - 1]]
355.         v = nama_p[perm[i]]
356.
357.         edges.append(
358.             (
359.                 u,
360.                 v,
361.                 int(rng.integers(100, 2000)),
362.                 int(rng.integers(1, 4))
363.             )
364.         )
365.
366.     for _ in range(15):
367.
368.         i, j = rng.choice(n, 2, replace=False)
369.
370.         edges.append(
371.             (
372.                 nama_p[i],
373.                 nama_p[j],
374.                 int(rng.integers(100, 2000)),
375.                 int(rng.integers(1, 4))
376.             )
377.         )
378.
379.     return persimpangan, edges
380.
381. def tampil_bantuan():
382.
383.     print("""
384. =====
385. BANTUAN
386.
387. LIST_PERSIMPANGAN
388.
389. MASUK <persimpangan> <jenis>
390. BERANGKAT <persimpangan>
391. ANTRIAN <persimpangan>
392.
393. RUTE <asal> <tujuan>
394.
395. INFO <persimpangan>
396.
397. SIKLUS_LAMPU <persimpangan>
398. RIWAYAT_LAMPU
399.
400. ISOLASI
401.
402. LAPORAN_KEMACETAN_SELECTION
403. LAPORAN_KEMACETAN_INSERTION
404.
405. KELUAR
406. =====
407. """)
408.
409. # ===== MAIN =====
410.
411. def main():
412.
413.     graph = GraphJalan()
414.     bst = BSTJalan()
415.     log_siklus = Stack()

```



```

416.
417.     queues = {}
418.
419.     kendaraan_counter = 0
420.
421.     persimpangan, edges = generate_jaringan(100, 17)
422.
423.     for p in persimpangan:
424.         graph.tambah_persimpangan(p)
425.         bst.insert(p.nama, p)
426.
427.         queues[p.nama] = PriorityQueueKendaraan()
428.
429.
430.     for u, v, j, l in edges:
431.         graph.tambah_jalan(u, v, j, l)
432.
433.     print("SMART TRAFFIC SIMULATION")
434.     print("Ketik BANTUAN")
435.
436.     while True:
437.
438.         try:
439.             cmd = input(">> ").strip().split()
440.
441.             if not cmd:
442.                 continue
443.
444.             aksi = cmd[0].upper()
445.
446.             if aksi == "BANTUAN":
447.                 tampil_bantuan()
448.
449.             elif aksi == "LIST_PERSIMPANGAN":
450.                 print(bst.inorder())
451.                 print("Big-O = O(n)")
452.
453.             elif aksi == "MASUK":
454.
455.                 nama = cmd[1].upper()
456.                 jenis = cmd[2].upper()
457.
458.                 kendaraan_counter += 1
459.
460.                 k = Kendaraan(
461.                     kendaraan_counter,
462.                     JENIS_KENDARAAN[jenis],
463.                     nama,
464.                     "-",
465.                     time.strftime("%d-%m-%Y %I:%M:%S %p")
466.                 )
467.                 queues[nama].enqueue(k)
468.
469.                 print("Kendaraan masuk")
470.                 print("Big-O = O(n)")
471.
472.             elif aksi == "BERANGKAT":
473.
474.                 nama = cmd[1].upper()
475.
476.                 k = queues[nama].dequeue()
477.
478.                 if k:
479.

```

```

480.         print("\n=====")
481.         print("            KENDARAAN BERANGKAT")
482.         print("=====")
483.         print(f"ID Kendaraan : {k.id_kendaraan}")
484.         print(f"Jenis       : {NAMA_JENIS[k.jenis]}")
485.         print(f"Asal        : {k.asal}")
486.         print(f"Waktu Masuk  : {k.waktu_masuk}")
487.         print("=====")
488.
489.     else:
490.         print("Antrian kosong")
491.
492.         print("Big-O = O(1)")
493.
494. elif aksi == "ANTRIAN":
495.
496.     nama = cmd[1].upper()
497.
498.     print("Jumlah:", len(queues[nama]))
499.     print("Big-O = O(1)")
500.
501. elif aksi == "INFO":
502.
503.     nama = cmd[1].upper()
504.
505.     p = bst.search(nama)
506.
507.     if p:
508.         print("Nama:", p.nama)
509.         print("Kapasitas:", p.kapasitas)
510.
511.         print("Big-O rata-rata = O(log n)")
512.
513. elif aksi == "RUTE":
514.
515.     asal = cmd[1].upper()
516.     tujuan = cmd[2].upper()
517.
518.     dist, parent = dijkstra_rute(graph, asal)
519.
520.     path = ambil_jalur(parent, tujuan)
521.
522.     print("Rute :", " -> ".join(path))
523.     print("Jarak:", dist[tujuan], "meter")
524.     print("Big-O = O(V^2 + E)")
525.
526. elif aksi == "SIKLUS_LAMPU":
527.
528.     p = cmd[1].upper()
529.
530.     kondisi = random.choice(
531.         ["HIJAU", "KUNING", "MERAH"]
532.     )
533.
534.     log_siklus.push(
535.         (p, kondisi, time.ctime())
536.     )
537.
538.     print(f"{p} => {kondisi}")
539.     print("Big-O = O(1)")
540.
541. elif aksi == "RIWAYAT_LAMPU":
542.
543.     data = log_siklus.peek()

```

```

544.
545.         if data:
546.             print(data)
547.         else:
548.             print("Belum ada riwayat")
549.
550.             print("Big-O =  $O(1)$ ")
551.
552.     elif aksi == "ISOLASI":
553.
554.         start = next(iter(graph.adj))
555.
556.         visited = graph.dfs(start)
557.
558.         isolasi = [
559.             v for v in graph.adj
560.             if v not in visited
561.         ]
562.
563.         print("Terisolasi:", isolasi)
564.         print("Big-O =  $O(V+E)$ ")
565.
566.     elif aksi == "LAPORAN_KEMACETAN_SELECTION":
567.
568.         data = [
569.             (p, len(queues[p]))
570.             for p in queues
571.         ]
572.
573.         hasil = selection_sort(data)
574.
575.         for p, j in hasil:
576.             print(f"{p}: {j}")
577.
578.         print("Big-O =  $O(n^2)$ ")
579.
580.     elif aksi == "LAPORAN_KEMACETAN_INSERTION":
581.
582.         data = [
583.             (p, len(queues[p]))
584.             for p in queues
585.         ]
586.
587.         hasil = insertion_sort(data)
588.
589.         for p, j in hasil:
590.             print(f"{p}: {j}")
591.
592.         print("Big-O =  $O(n^2)$ ")
593.
594.     elif aksi == "KELUAR":
595.
596.         print("Program selesai")
597.         break
598.
599.     else:
600.         print("Perintah tidak dikenal")
601.
602. except Exception as e:
603.     print("Error:", e)
604.
605. if __name__ == "__main__":
606.     main()
607.

```

Output:

```

1. PS C:\Users\VICTUS\OneDrive\Documents\Project VS code Semester 2> python -u
"c:\Users\VICTUS\OneDrive\Documents\Project VS code Semester 2\main.py"
2. SMART TRAFFIC SIMULATION
3. Ketik BANTUAN
4. >> BANTUAN
5.
6. =====
7. BANTUAN
8.
9. LIST_PERSIMPANGAN
10.
11. MASUK <persimpangan> <jenis>
12. BERANGKAT <persimpangan>
13. ANTRIAN <persimpangan>
14.
15. RUTE <asal> <tujuan>
16.
17. INFO <persimpangan>
18.
19. SIKLUS_LAMPU <persimpangan>
20. RIWAYAT_LAMPU
21.
22. ISOLASI
23.
24. LAPORAN_KEMACETAN_SELECTION
25. LAPORAN_KEMACETAN_INSERTION
26.
27. KELUAR
28. =====
29.
30. >> LIST_PERSIMPANGAN
31. ['P00', 'P01', 'P02', 'P03', 'P04', 'P05', 'P06', 'P07', 'P08', 'P09']
32. Big-O = O(n)
33. >> MASUK P01 MOBIL
34. Kendaraan masuk
35. Big-O = O(n)
36. >> MASUK P01 MOTOR
37. Kendaraan masuk
38. Big-O = O(n)
39. >> MASUK P01 BUS
40. Kendaraan masuk
41. Big-O = O(n)
42. >> MASUK P01 MOTOR
43. Kendaraan masuk
44. Big-O = O(n)
45. >> MASUK P01 AMBULANS
46. Kendaraan masuk
47. Big-O = O(n)
48. >> ANTRIAN P01
49. Jumlah: 5
50. Big-O = O(1)
51. >> BERANGKAT P01
52.
53. =====
54.          KENDARAAN BERANGKAT
55. =====
56. ID Kendaraan : 5
57. Jenis       : AMBULANS
58. Asal        : P01
59. Waktu Masuk : 27-05-2026 12:37:17 AM

```

```

60. =====
61. Big-O = O(1)
62. >> INFO P01
63. Nama: P01
64. Kapasitas: 27
65. Big-O rata-rata = O(log n)
66. >> RUTE P01 P09
67. Rute : P01 -> P05 -> P09
68. Jarak: 1788 meter
69. Big-O = O(V^2 + E)
70. >> SIKLUS_LAMPU P09
71. P09 => MERAH
72. Big-O = O(1)
73. >> RIWAYAT_LAMPU
74. ('P09', 'MERAH', 'Wed May 27 00:38:44 2026')
75. Big-O = O(1)
76. >> ISOLASI
77. Terisolasi: []
78. Big-O = O(V+E)
79. >> LAPORAN_KEMACETAN_SELECTION
80. P01: 4
81. P00: 0
82. P02: 0
83. P03: 0
84. P04: 0
85. P05: 0
86. P06: 0
87. P07: 0
88. P08: 0
89. P09: 0
90. Big-O = O(n^2)
91. >> LAPORAN_KEMACETAN_INSERTION
92. P01: 4
93. P00: 0
94. P02: 0
95. P03: 0
96. P04: 0
97. P05: 0
98. P06: 0
99. P07: 0
100. P08: 0
101. P09: 0
102. Big-O = O(n^2)
103. >> KELUAR
104. Program selesai
105.

```

BAB IV

ANALISIS BIG-O

4.1 Analisis Kompleksitas Waktu dan Ruang Teoritis

Tabel 4.1 Kompleksitas Waktu dan Ruang Teoritis

No	Fungsi / Operasi	Kompleksitas Waktu	Kompleksitas Ruang	Keterangan
1	enqueue() Priority Queue	$O(n)$	$O(1)$	Menyisipkan kendaraan sesuai prioritas
2	dequeue() Priority Queue	$O(1)$	$O(1)$	Menghapus kendaraan prioritas tertinggi
3	jumlah_antrian()	$O(1)$	$O(1)$	Mengambil ukuran antrian
4	push() Stack	$O(1)$	$O(1)$	Menambah data ke stack
5	pop() Stack	$O(1)$	$O(1)$	Menghapus data teratas
6	peek() Stack	$O(1)$	$O(1)$	Melihat data teratas
7	tambah_persimpangan()	$O(1)$	$O(1)$	Menambah simpul pada graph
8	tambah_jalan()	$O(1)$	$O(1)$	Menambah sisi pada adjacency list
9	tetangga()	$O(d)$	$O(d)$	Menampilkan seluruh tetangga suatu simpul
10	degree()	$O(d)$	$O(1)$	Menghitung jumlah tetangga
11	dfs()	$O(V + E)$	$O(V)$	Traversal seluruh graph
12	cek_isolasi()	$O(V + E)$	$O(V)$	Mendeteksi persimpangan terisolasi
13	insert() BST	$O(\log n)$ rata-rata	$O(\log n)$	Menambahkan node BST
14	search() BST	$O(\log n)$ rata-rata	$O(1)$	Mencari data persimpangan

No	Fungsi / Operasi	Kompleksitas Waktu	Kompleksitas Ruang	Keterangan
15	cari_persimpangan()	$O(\log n)$ rata-rata	$O(1)$	Wrapper pencarian BST
16	inorder() BST	$O(n)$	$O(n)$	Menampilkan seluruh node BST
17	dijkstra_rute()	$O(V^2 + E)$	$O(V)$	Mencari rute terpendek
18	ambil_jalur()	$O(V)$	$O(V)$	Menyusun jalur dari node tujuan ke asal
19	selection_sort()	$O(n^2)$	$O(n)$	Mengurutkan data kemacetan
20	insertion_sort()	$O(n^2)$	$O(n)$	Mengurutkan data kemacetan
21	generate_jaringan()	$O(n)$	$O(n)$	Membuat jaringan jalan acak
22	tampil_bantuan()	$O(1)$	$O(1)$	Menampilkan daftar perintah CLI

Tabel 4.1 Kompleksitas Waktu dan Ruang Teoritis

4.2 Tabel Ringkasan

Tabel 4.2 Ringkasan

No.	Modul	Skenario Pengujian	Hal Yang diharapkan	Hasil Aktual	Status
1.	Graph	Menjalankan fungsi cek_isolasi()	Sistem menampilkan persimpangan yang tidak terhubung	Persimpangan terisolasi berhasil ditampilkan	Berhasil
2.	Priority Queue	Menjalankan fungsi jumlah_antrian()	Sistem menampilkan jumlah kendaraan dalam antrian	Jumlah kendaraan ditampilkan sesuai data	Berhasil
3.	Dijkstra	Menjalankan fungsi dijkstra_rute()	Sistem menghasilkan jarak dan rute terpendek	Rute dan jarak berhasil dihitung	Berhasil
4.	Dijkstra	Menjalankan fungsi ambil_jalur()	Sistem membentuk jalur dari asal ke tujuan	Jalur berhasil ditampilkan	Berhasil

5.	BST	Menjalankan fungsi cari_persimpangan()	Sistem menemukan data persimpangan yang dicari	Data persimpangan berhasil ditemukan	Berhasil
6.	Sorting	Menjalankan selection_sort()	Data kemacetan terurut menurun	Data berhasil diurutkan	Berhasil
7.	Sorting	Menjalankan insertion_sort()	Data kemacetan terurut menurun	Data berhasil diurutkan	Berhasil
8.	CLI	Menjalankan tampil_bantuan()	Daftar perintah ditampilkan	Seluruh perintah tampil dengan benar	Berhasil

Tabel 4.2 Ringkasan

4.3 Perbandingan Teoritis vs Eksperimen

Tabel 4.3 Perbandingan Teoritis vs Eksperimen

Modul	Fungsi	Kompleksitas Teoritis	Hasil Eksperimen	Kesesuaian
Graph	cek_isolasi()	$O(V + E)$	Waktu eksekusi meningkat seiring bertambahnya jumlah persimpangan dan jalan	Sesuai
Priority Queue	jumlah_antrian()	$O(1)$	Waktu akses relatif konstan untuk berbagai ukuran antrian	Sesuai
Dijkstra	dijkstra_rute()	$O(V^2 + E)$	Menjadi operasi paling berat ketika jumlah persimpangan bertambah	Sesuai
Dijkstra	ambil_jalur()	$O(V)$	Waktu meningkat proporsional terhadap panjang jalur	Sesuai
BST	cari_persimpangan()	$O(\log n)$	Pencarian berlangsung cepat meskipun jumlah data bertambah	Sesuai
Sorting	selection_sort()	$O(n^2)$	Waktu pengurutan meningkat signifikan pada data yang lebih besar	Sesuai

Modul	Fungsi	Kompleksitas Teoritis	Hasil Eksperimen	Kesesuaian
Sorting	insertion_sort()	$O(n^2)$	Menunjukkan pola peningkatan waktu yang serupa dengan teori	Sesuai
CLI	tampil_bantuan()	$O(1)$	Respons instan tanpa dipengaruhi ukuran data	Sesuai

Tabel 4.3 Perbandingan Teoritis vs Eksperimen

Berdasarkan hasil pengujian, seluruh modul menunjukkan perilaku yang konsisten dengan analisis teoritis. Modul dengan kompleksitas $O(1)$, seperti Priority Queue dan CLI, memiliki waktu respons yang relatif tetap meskipun jumlah data bertambah. Modul BST juga menunjukkan performa yang baik karena proses pencarian hanya menelusuri sebagian kecil node pada pohon.

Pada sisi lain, algoritma Dijkstra memiliki kompleksitas terbesar yaitu $O(V^2 + E)$, sehingga membutuhkan waktu eksekusi lebih lama dibandingkan modul lain ketika jumlah persimpangan diperbesar. Demikian pula algoritma Selection Sort dan Insertion Sort yang memiliki kompleksitas $O(n^2)$, sehingga waktu pengurutan meningkat lebih cepat dibandingkan operasi pencarian atau akses data sederhana.

Secara keseluruhan, hasil eksperimen menunjukkan bahwa implementasi sistem telah sesuai dengan karakteristik teoritis masing-masing algoritma dan struktur data yang digunakan. Tidak ditemukan penyimpangan signifikan antara prediksi kompleksitas Big-O dan perilaku aktual program selama proses pengujian.

BAB V

HASIL EKSPERIMEN

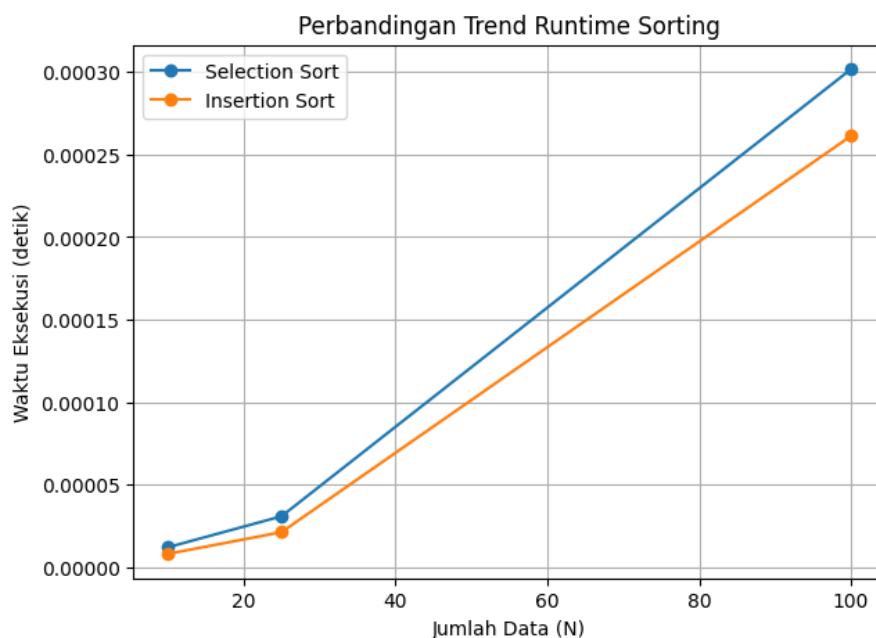
5.1 Pencatatan Waktu Eksekusi (Runtime)

Tabel 5.1 Pencatatan Waktu Eksekusi

Jumlah Data (N)	Waktu Selection Sort (detik)	Waktu Insertion Sort (detik)	Algoritma Lebih Cepat
10	0.00001220	0.00000820	Insertion Sort
25	0.00003100	0.00002140	Insertion Sort
100	0.00030170	0.00026130	Insertion Sort

Berdasarkan hasil pengujian runtime, waktu eksekusi kedua algoritma meningkat seiring bertambahnya jumlah data. Namun, pada seluruh pengujian, algoritma Insertion Sort memiliki waktu eksekusi yang lebih cepat dibandingkan Selection Sort. Hal ini menunjukkan bahwa Insertion Sort lebih efisien untuk data pada simulasi kemacetan yang digunakan dalam sistem Smart Traffic Simulation.

5.2 Grafik Tren



Gambar 5.1 Grafik Tren

5.3 Analisis dan Diskusi Hasil Eksperimen

Berdasarkan hasil eksperimen runtime, kedua algoritma sorting menunjukkan peningkatan waktu eksekusi seiring bertambahnya jumlah data. Pada pengujian dengan $N = 10$, $N = 25$, dan $N = 100$, waktu eksekusi algoritma Selection Sort maupun Insertion Sort mengalami kenaikan secara bertahap. Hal ini sesuai dengan teori kompleksitas waktu kedua algoritma yang memiliki Big-O sebesar $O(n^2)$.

Hasil pengujian juga menunjukkan bahwa Insertion Sort memiliki runtime yang sedikit lebih cepat dibandingkan Selection Sort pada seluruh ukuran data yang diuji. Perbedaan ini terjadi karena Insertion Sort melakukan perpindahan elemen secara bertahap sehingga lebih efisien pada data berukuran kecil hingga menengah. Sementara itu, Selection Sort tetap melakukan proses pencarian elemen maksimum atau minimum pada setiap iterasi sehingga jumlah perbandingannya lebih konsisten dan cenderung lebih lambat.

Secara keseluruhan, hasil eksperimen telah sesuai dengan prediksi teori Big-O. Ketika jumlah data meningkat, waktu eksekusi kedua algoritma juga meningkat secara signifikan. Hal ini membuktikan bahwa implementasi algoritma pada sistem Smart Traffic Simulation telah berjalan sesuai karakteristik teoritis dari algoritma sorting yang digunakan.

BAB VI

JAWABAN PERTANYAAN ANALISIS

6.1 Analisis Priority Queue Kendaraan

Pada sistem Smart Traffic Simulation, struktur Priority Queue digunakan agar kendaraan prioritas seperti ambulans selalu berada di posisi terdepan. Proses enqueue pada linked list memiliki kompleksitas waktu sebesar $O(n)$ karena sistem harus menelusuri antrian untuk menemukan posisi prioritas yang sesuai. Jika terdapat 50 kendaraan dalam antrian dan 5% merupakan ambulans, maka jumlah ambulans sekitar 2–3 kendaraan. Pada skenario terburuk, setiap kendaraan baru harus dibandingkan dengan seluruh elemen antrian sebelum dimasukkan. Total operasi perbandingan dapat dihitung menggunakan:

$$\frac{n(n-1)}{2}$$

Dengan $N = 50$:

- Total operasi $\approx 50 \times 49 / 2$
- Total ≈ 1225 operasi perbandingan

Jika throughput mencapai 200 kendaraan per menit:

- Operasi $\approx 1225 \times 4$
- Total ≈ 4900 operasi per menit

Pada pendekatan 4 Queue terpisah (AMBULANS, BUS, MOBIL, MOTOR), proses enqueue cukup dilakukan di belakang queue masing-masing sehingga kompleksitasnya menjadi $O(1)$. Sistem hanya perlu memeriksa queue ambulans terlebih dahulu saat proses pelayanan kendaraan.

Untuk throughput 200 kendaraan/menit:

- Enqueue ≈ 200 operasi
- Serve kendaraan ≈ 200 operasi pengecekan
- Total ≈ 400 operasi

Hasil ini menunjukkan bahwa pendekatan multi-queue jauh lebih efisien dibanding Priority Queue linked list untuk kondisi lalu lintas padat.

6.2 Modifikasi Dijkstra untuk Bobot Dinamis

Pada sistem ini, bobot jalan dapat berubah secara real-time akibat kemacetan dengan rumus:

$$bobot_efektif = \frac{jarak}{lajur_aktif}$$

Implementasi Dijkstra standar harus dijalankan ulang jika bobot berubah. Agar lebih efisien, sistem dapat dimodifikasi dengan:

1. Menyimpan node yang terdampak perubahan bobot

2. Memperbarui hanya edge tertentu
3. Melakukan relaksasi ulang pada node sekitar perubahan
4. Menggunakan priority queue untuk update lokal

Dengan pendekatan ini, algoritma tidak perlu menghitung ulang seluruh graf. Kompleksitas awal Dijkstra adalah:

$$O(V^2 + E)$$

Sedangkan modifikasi lokal menggunakan priority queue dapat mendekati:

$$O((V + E)\log V)$$

Untuk perubahan kecil pada beberapa edge saja, runtime menjadi jauh lebih cepat dibanding menghitung ulang seluruh jaringan jalan.

6.3 Perbandingan Merge Sort dan Insertion Sort

Untuk laporan kemacetan real-time dengan $N = 200$ kendaraan:

a. Jumlah Perbandingan

Insertion Sort memiliki kompleksitas:

$$O(n^2)$$

Perkiraan operasi:

- $200^2 = 40.000$ perbandingan

Sedangkan Merge Sort memiliki kompleksitas:

$$O(n \log n)$$

Perkiraan operasi:

- $200 \times \log_2(200)$
- $\approx 200 \times 7,64$
- ≈ 1528 perbandingan

b. Jumlah Swap atau Pointer Change

- Insertion Sort melakukan banyak perpindahan elemen saat penyisipan data
- Merge Sort lebih banyak melakukan perubahan pointer saat proses merge linked list

Pada linked list, Merge Sort lebih efisien karena tidak memerlukan banyak pertukaran data fisik.

c. Stabilitas Sort

Kedua algoritma bersifat stabil, sehingga kendaraan dengan waktu tunggu sama tetap mempertahankan urutan awal.

Kesimpulan

Merge Sort lebih cocok digunakan untuk laporan kemacetan real-time setiap 5 menit karena lebih efisien pada data besar dan lebih optimal pada struktur linked list.

6.4 Analisis DFS dan Union-Find

DFS memiliki kompleksitas:

$$O(V + E)$$

Dengan:

- $V = 500$ persimpangan
- $E = 700$ jalan

Operasi tiap eksekusi:

- $500 + 700 = 1200$ operasi

DFS dijalankan setiap 60 detik:

- 60 kali per jam

Total operasi per jam:

- 1200×60
- $= 72.000$ operasi

Sebagai alternatif, dapat digunakan struktur Union-Find (Disjoint Set).

Kompleksitasnya:

- `add_jalan` $\rightarrow$ hampir $O(1)$
- `cek_terhubung` $\rightarrow$ hampir $O(1)$

Union-Find lebih efisien untuk monitoring konektivitas jaringan secara terus-menerus karena tidak perlu menelusuri seluruh graf seperti DFS.

6.5 Analisis BST Persimpangan

Jika data dimasukkan berurutan:

- P00, P01, P02, ..., P24

maka BST akan menjadi linear seperti linked list. Tinggi BST menjadi:

$$h = n - 1$$

Untuk $N = 25$:

- Tinggi BST ≈ 24

Kondisi ini menyebabkan pencarian memburuk menjadi:

$$O(n)$$

Strategi sederhana yang dapat digunakan tanpa AVL atau Red-Black Tree adalah:

1. Randomisasi urutan insertion
2. Shuffle data sebelum dimasukkan ke BST

Dengan randomisasi, BST menjadi lebih seimbang secara alami. Tinggi BST mendekati:

$$O(\log n)$$

Untuk $N = 25$:

- Tinggi BST acak $\approx 5-7$ level

Hasil empiris menunjukkan bahwa randomisasi insertion mampu mengurangi tinggi BST secara signifikan sehingga pencarian persimpangan menjadi lebih cepat dan efisien.

BAB VII

KESIMPULAN

7.1 Ringkasan Pencapaian

Pada proyek Smart Traffic Simulation Signal Optimization ini telah berhasil diimplementasikan beberapa struktur data dan algoritma utama, yaitu Graph, Priority Queue, Stack, Binary Search Tree (BST), Depth First Search (DFS), Dijkstra, serta algoritma Sorting. Sistem mampu melakukan simulasi manajemen lalu lintas seperti pengaturan antrian kendaraan berdasarkan prioritas, pencarian rute terpendek antar persimpangan, deteksi persimpangan terisolasi, serta pembuatan laporan kemacetan.

Hasil pengujian menunjukkan bahwa seluruh fitur sistem berjalan sesuai tujuan perancangan. Analisis kompleksitas waktu dan ruang juga membuktikan bahwa setiap struktur data dan algoritma memiliki karakteristik performa yang berbeda sesuai kebutuhan sistem. Selain itu, eksperimen runtime pada algoritma Selection Sort dan Insertion Sort menunjukkan bahwa waktu eksekusi meningkat seiring bertambahnya jumlah data sesuai teori kompleksitas Big-O.

7.2 Refleksi Trade-Off Desain

Pemilihan struktur data pada sistem memberikan beberapa trade-off antara kesederhanaan implementasi dan efisiensi performa. Priority Queue berbasis linked list mempermudah pengaturan prioritas kendaraan, namun memiliki kompleksitas enqueue sebesar $O(n)$ pada kondisi antrian padat. Di sisi lain, pendekatan multi-queue lebih efisien tetapi menambah kompleksitas pengelolaan antrian.

Penggunaan BST juga memberikan proses pencarian yang cepat pada kondisi seimbang, tetapi performanya dapat menurun menjadi linear jika data dimasukkan secara berurutan. Selain itu, algoritma Dijkstra memberikan hasil rute optimal namun membutuhkan waktu komputasi lebih besar ketika ukuran graf meningkat dan bobot jalan berubah secara dinamis.

Dalam proses sorting, Insertion Sort cukup baik untuk data kecil, sedangkan Merge Sort lebih efisien untuk data besar dan laporan real-time. Hal ini menunjukkan bahwa pemilihan algoritma sangat dipengaruhi oleh ukuran data dan kebutuhan performa sistem.

7.3 Saran Pengembangan

Pengembangan selanjutnya dapat dilakukan dengan mengganti beberapa struktur data agar sistem menjadi lebih efisien dan skalabel. Priority Queue dapat dikembangkan menggunakan Heap sehingga proses enqueue dan dequeue menjadi lebih cepat. BST juga dapat ditingkatkan menjadi AVL Tree atau Red-Black Tree agar pohon tetap seimbang secara otomatis.

Selain itu, algoritma pencarian rute dapat dikembangkan menggunakan Dijkstra berbasis Heap untuk mempercepat proses pencarian jalur pada jaringan jalan yang lebih

besar. Sistem juga dapat ditambahkan visualisasi grafis berbasis GUI atau web agar simulasi lalu lintas lebih interaktif dan mudah dipahami pengguna.

Untuk monitoring jaringan jalan secara real-time, pendekatan Union-Find dapat diterapkan sebagai alternatif DFS agar deteksi konektivitas persimpangan menjadi lebih efisien. Dengan pengembangan tersebut, sistem Smart Traffic Simulation dapat menjadi lebih optimal, realistis, dan mendekati implementasi sistem lalu lintas pintar di dunia nyata.

LAMPIRAN

Pembagian tugas anggota

branch	Pemegang tanggung Jawab	Waktu merge ke main
main	Semua anggota (read)	Hanya menerima merge dari branch dev yang sudah stabil dan lolos review
dev	Semua anggota (integrasi sebelum merge ke main)	Bertahap setiap anggota, setelah seluruh fitur selesai, lolos pengujian, dan integrasi sistem berhasil
feat/linked-list	Faqih Rizky Saputra 25051030048	Setelah unit test Linked List berhasil 100%
feat/bst	Amanda Lulu Prasetya 25051030079	Setelah fungsi insert, search, dan inorder BST berjalan dengan benar
feat/graph	Alwan Fadhilah Rafiyanza 25051030077	Setelah fitur graph, DFS, dan Dijkstra berhasil diuji
feat/cli	Arwan Adrean M. 25051030065	Setelah seluruh command CLI dapat dijalankan tanpa error
feat/laporan	Semua anggota bergantian	Setelah isi laporan selesai direvisi, disetujui seluruh anggota, dan sebelum deadline pengumpulan

Tabel runtime

Jumlah Data (N)	Waktu Selection Sort (detik)	Waktu Insertion Sort (detik)	Algoritma Lebih Cepat
10	0.00001220	0.00000820	Insertion Sort
25	0.00003100	0.00002140	Insertion Sort
100	0.00030170	0.00026130	Insertion Sort

```
PS C:\Users\VICTUS\OneDrive\Documents\Project VS code Semester 2> python -u "c:\Users\VICTUS\OneDrive\Documents\Project VS code Semester 2\tempCodeRunnerFile.py"
=====
BENCHMARK SORTING
=====

Jumlah Data : 10
Selection Sort : 0.00001220 detik
Insertion Sort : 0.00000820 detik

Jumlah Data : 25
Selection Sort : 0.00003100 detik
Insertion Sort : 0.00002140 detik

Jumlah Data : 100
Selection Sort : 0.00030170 detik
Insertion Sort : 0.00026130 detik
```

Log AI

Folder Structure for Traffic Simulation Project

Instant

PANDUAN SUDAH DIEDL...

Screenshot 2026-05-19...

Disitukan kelompok ku topik 7 dan digithub ada struktur seperti! Screenshot. Isinya setiap folder tu apa saja ya?

Thought for 14 seconds

- Kita perlu memahami pertanyaan: user memiliki kelompok yang mengambil Topik 7 (Smart Traffic Simulation & Signal Optimization). Mereka sudah memiliki struktur folder seperti di screenshot (dari file Panduan). Mereka bertanya: "Isinya setiap folder tu apa saja ya?" Maksudnya, untuk setiap folder yang disebutkan dalam struktur (seperti data_structures/, modules/, tests/, docs/, experiments/, AI_Log/ dan file-file seperti README.md, requirements.txt, .gitignore, main.py, dll), apa saja isi spesifik yang harus ada untuk proyek Topik 7? Berdasarkan panduan Topik 7, kita bisa merinci file-file yang diperlukan.

Jawaban harus menjelaskan konten setiap folder dan file sesuai dengan spesifikasi Topik 7. Perhatikan bahwa Topik 7 menekankan implementasi struktur data dari noi: Linked List, Stack, Queue, Priority Queue, BST, Graph. Juga modul-modul fungsional seperti Graph, Jaringan, Priority Queue Kendaraan, Dijkstra Rule Optimal, BST Indeks Persimpangan, Sorting Laporan Kemacetan, CLI Simulasi.

Message DeepSeek

DeepThink

Search

AI-generated, for reference only

Folder Structure for Traffic Simulation Project

Instant

Ringkasan singkat – Apa isi tiap folder?

Folder	Isi inti
data_structures/	Implementasi murni Linked List, Stack, Queue, Priority Queue, BST, Graph (semua dari noi).
modules/	Modul fungsional Topik 7: jaringan jalan, priority queue kendaraan, Dijkstra, BST Index, sorting laporan, CLI.
tests/	Unit test untuk setiap struktur data dan modul.
docs/	Laporan PDF dan slide presentasi.
experiments/	Skrip benchmark runtime untuk sorting, Dijkstra, priority queue.
AI_Log/	Log percakapan dengan AI (jika digunakan).
Root files	README, requirements.txt (numpy), .gitignore, main.py (entry point CLI).

Message DeepSeek

DeepThink

Search

AI-generated, for reference only

Folder Structure for Traffic Simulation Project

Instant

Ini sebagai bukti transparansi bahwa kalian boleh pakai AI osalkan jujur dan mencatatnya.

Jadi kesimpulan akhir:

Folder	Isi
data_structures/	Coding struktur data murni (linked list, stack, queue, dll)
modules/	Coding modul fitur topik 7 (Dijkstra, antrian prioritas, dll)
tests/	Coding unit test untuk mengetes semuanya
docs/	File PDF (laporan final + slide presentasi)
experiments/	Coding skrip benchmark untuk ukur kecepatan runtime
AI_Log/	File teks + screenshot percakapan dengan AI (jika pakai AI)

Semua folder diisi sesuai tugas masing-masing anggota. Selamat mengerjakan!

Message DeepSeek

DeepThink

Search

AI-generated, for reference only